

What is MLflow?



"An open source platform for the machine learning lifecycle" - [MLflow.org](https://mlflow.org)

¹ www.mlflow.org

Components of MLflow

- **MLflow Tracking:**
 - Record metrics and parameters from training runs
 - Query data from experiments
 - Store models, artifacts and code
- **Model Registry:**
 - Store and version ML models
 - Load and deploy ML models
- **MLflow Models:**
 - Standardize models for deployment
 - Build customized models
- **MLflow Projects:**
 - Package ML code for reproducibility
 - Package ML code for repeatability

MLflow experiments

Experiments



Search Experiments

- ☒ Default  
- ☐ Scores Experiment  
- ☐ Scores  
- ☐ Unicorn Experiment  
- ☐ Unicorn  
- ☐ Unicorn Model  
- ☐ 5  
- ☐ Test  
- ☐ 7  
- ☐ Test 2  
- ☐ 9  
- ☐ Test 3  
- ☐ 11  

Default

Share

 Track machine learning training runs in experiments. [Learn more](#) 

Experiment ID: 0 Artifact Location: ./mlruns/0

> Description [Edit](#)

 metrics.rmse < 1 and params.model = "tree" 

 Sort: Created 

 Columns 



 Refresh

Time created: All time 

State: Active 

Showing 3 matching runs

				Metrics	
☐	Run Name	Created 	Duration	metric_1	metric_2
☐	rumbling-deer-742	 3 months ago	2.0s	0.872	1.824
☐	receptive-kit-255	 3 months ago	2.0s	0.86	1.356
☐	bright-gnu-469	 3 months ago	2.0s	0.242	1.263

Starting a new experiment

```
import mlflow
# Create new Experiment
mlflow.create_experiment("My Experiment")
# Tag new experiment
mlflow.set_experiment_tag("scikit-learn", "lr")
# Set the experiment
mlflow.set_experiment("My Experiment")
```

Starting a training run

```
import mlflow

# Start a run
mlflow.start_run()
```

```
<ActiveRun: >
```

```
# End a run
mlflow.end_run()
```

Logging to MLflow Tracking

- **Metrics**

- `log_metric("accuracy", 0.90)`
- `log_metrics({"accuracy": 0.90, "loss": 0.50})`

- **Parameters**

- `log_param("n_jobs", 1)`
- `log_params({"n_jobs": 1, "fit_intercept": False})`

- **Artifacts**

- `log_artifact("file.py")`
- `log_artifacts("./directory/")`

Autolog

```
# Automatically log model and metrics  
mlflow.FLAVOR.autolog()
```

```
# Scikit-learn built-in flavor  
mlflow.sklearn.autolog()
```


Built-In Flavors

- Python Function (python_function)
- R Function (crate)
- H₂O (h2o)
- Keras (keras)
- MLeap (mleap)
- PyTorch (pytorch)
- Scikit-learn (sklearn)
- Spark MLlib (spark)
- TensorFlow (tensorflow)
- ONNX (onnx)
- MXNet Gluon (gluon)
- XGBoost (xgboost)

- Write custom tools from ML libraries
- Flavors simplify the new for custom code

```
# Import flavor from mlflow module
import mlflow.FLAVOR
```

¹ mlflow.org

Open MLflow UI

```
# Open MLflow Tracking UI  
mlflow ui
```

Go to: <http://localhost:5000>



MLflow REST API



¹ istock.com

Model API functions

```
# Save a model to the local filesystem  
mlflow.sklearn.save_model(model, path)
```

```
# Log a model as an artifact to MLflow Tracking.  
mlflow.sklearn.log_model(model, artifact_path)
```

```
# Load a model from local filesystem or from MLflow Tracking.  
mlflow.sklearn.load_model(model_uri)
```

Load model

- Local Filesystem - `relative/path/to/local/model` or `/Users/me/path/to/local/model`
- MLflow Tracking - `runs:/<mlflow_run_id>/run-relative/path/to/model`
- S3 Support - `s3://my_bucket/path/to/model`

Custom Python models

- Built in Flavor - `python_function`
- `mlflow.pyfunc`
 - `save_model()`
 - `log_model()`
 - `load_model()`

Custom model class

- Custom model class
 - `MyClass(mlflow.pyfunc.PythonModel)`
- PythonModel class
 - `load_context()` - loads artifacts when `mlflow.pyfunc.load_model()` is called
 - `predict()` - takes model input and performs user defined evaluation

Example custom Class

```
import mlflow.pyfunc

# Define the model class
class CustomPredict(mlflow.pyfunc.PythonModel):
    # Load artifacts
    def load_context(self, context):
        self.model = mlflow.sklearn.load_model(context.artifacts["custom_model"])
    # Evaluate input using custom_function()
    def predict(self, context, model_input):
        prediction = self.model.predict(model_input)
        return custom_function(prediction)
```


Saving and logging a custom model

```
# Save model to local filesystem
```

```
mlflow.pyfunc.save_model(path="custom_model", python_model=CustomPredict())
```

```
# Log model to MLflow Tracking
```

```
mlflow.pyfunc.log_model(artifact_path="custom_model", python_model=CustomPredict())
```


Loading custom models

```
# Load model from local filesystem  
mlflow.pyfunc.load_model("local")
```

```
# Load model from MLflow Tracking  
mlflow.pyfunc.load_model("runs:/run_id/tracking_path")
```

Model Evaluation

- `mlflow.evaluate()` - Performance based on a dataset
- Regression and Classification models

Evaluation Example

```
# Training Data
X_train, X_test, y_train, y_test = \
    train_test_split(X, y,
                    train_size=0.7, random_state=0)

# Linear Regression model
lr = LinearRegression()
lr.fit(X_train, y_train)
```

```
# Dataset
eval_data = X_test
eval_data["test_label"] = y_test

# Evaluate model with Dataset
mlflow.evaluate(
    "runs:/run_id/model",
    eval_data,
    targets="test_label",
    model_type="regressor"
)
```

Tracking UI

▼ Artifacts

▼ model

- MLmodel
- conda.yaml
- model.pkl
- python_env.yaml
- requirements.txt
- estimator.html
- shap_beeswarm_plot.png
- shap_feature_importance_plot
- shap_summary_plot.png



SHAP

¹ shap.readthedocs.io

Serving Models

```
# MLflow serve command  
mlflow models serve --help  
Usage: mlflow models serve [OPTIONS]
```

Serve uri

Local Filesystem

```
mlflow models serve -m relative/path/to/local/model
```

Run ID

```
mlflow models serve -m runs:<mlflow_run_id>/artifacts/model
```

AWS S3

```
mlflow models serve -m s3://my_bucket/path/to/model
```

Serve example

```
# Serve model from run
mlflow models serve -m runs:/e84a122920de4bdeaedb54146deeb429/artifacts/model
```

```
2023/03/12 16:28:28 INFO mlflow.models.flavor_backend_registry:
Selected backend for flavor 'python_function'
2023/03/12 16:28:28 INFO mlflow.pyfunc.backend: === Running command
'exec gunicorn --timeout=60 -b 127.0.0.1:5000 -w 1 ${GUNICORN_CMD_ARGS} --
mlflow.pyfunc.scoring_server.wsgi:app'
[2023-03-12 16:28:29 -0400] [48431] [INFO] Starting gunicorn 20.1.0
[2023-03-12 16:28:29 -0400] [48431] [INFO] Listening at: http://127.0.0.1:5000
(48431)
[2023-03-12 16:28:29 -0400] [48431] [INFO] Using worker: sync
[2023-03-12 16:28:29 -0400] [48432] [INFO] Booting worker with pid: 48432
```


REST API

- `/ping` - for health checks
- `/health` - for health checks
- `/version` - for getting the version of MLflow
- `/invocations` - for model scoring
- Port 5000

Invocations endpoint

`/invocations`

No, Name, Subject

1, Bill Johnson, English

2, Gary Valentine, Mathematics

Content-Type : `application/json` or
`application/csv`

```
{
  "1": {
    "No": "1",
    "Name": "Bill Johnson",
    "Subject": "English"
  },
  "2": {
    "No": "2",
    "Name": "Gary Valentine",
    "Subject": "Mathematics"
  }
}
```

CSV format

- Pandas Dataframe
- `pandas_df.to_csv()`

JSON format

- `dataframe_split` - pandas DataFrame in split orientation
- `dataframe_records` - pandas DataFrame in records orientation

DataFrame split

```
# Dataframe split orientation
{
  "dataframe_split": {
    "columns": ["sex", "age", "weight"],
    "data": [["male", 23, 160], ["female", 33, 120]]
  }
}
```

Invocations Request

```
# Send dataframe_split orientation payload to MLflow
curl http://127.0.0.1:5000/invocations -H 'Content-Type: application/json' -d '{
  "dataframe_split": {
    "columns": ["sex", "age", "weight"],
    "data": [["male", 23, 160], ["female", 33, 120]]
  }
}'
```

```
{"predictions": [1, 0]}
```